



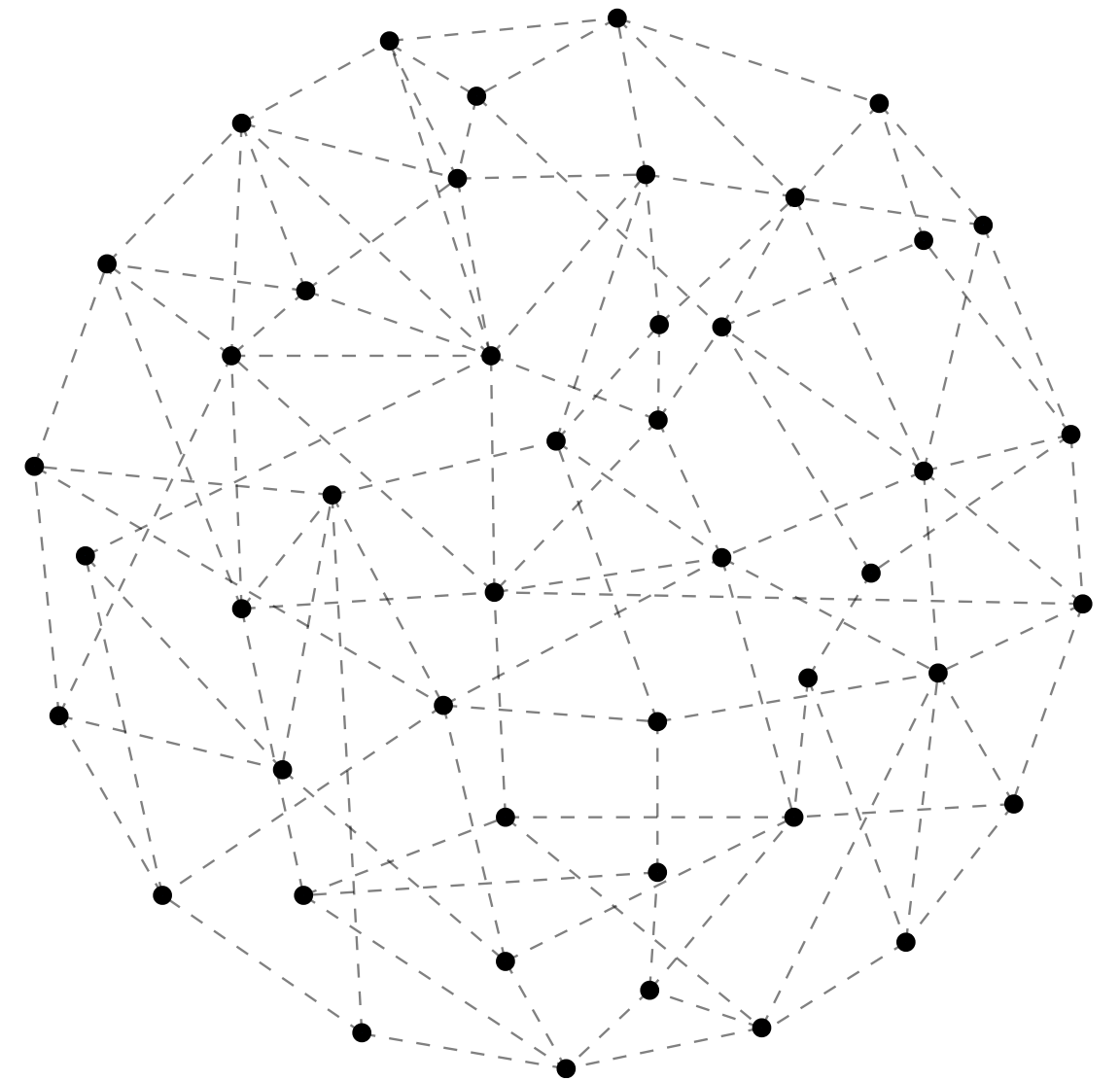
ELECTRICAL ENGINEERING ASSOCIATION

COMPRESSION OF DEEP NEURAL NETWORKS

Group -5

PREREQUISITES

- **Basic Machine Learning:** Basic knowledge of training and testing machine learning models is required
- **Neural Network Fundamentals:** Understanding neurons, layers, and weights in neural networks is helpful.
- **Model Size and Computation:** Larger networks have more parameters and need more memory and computation.
- **Hardware Awareness:** Embedded devices have limited memory, power, and processing capability.



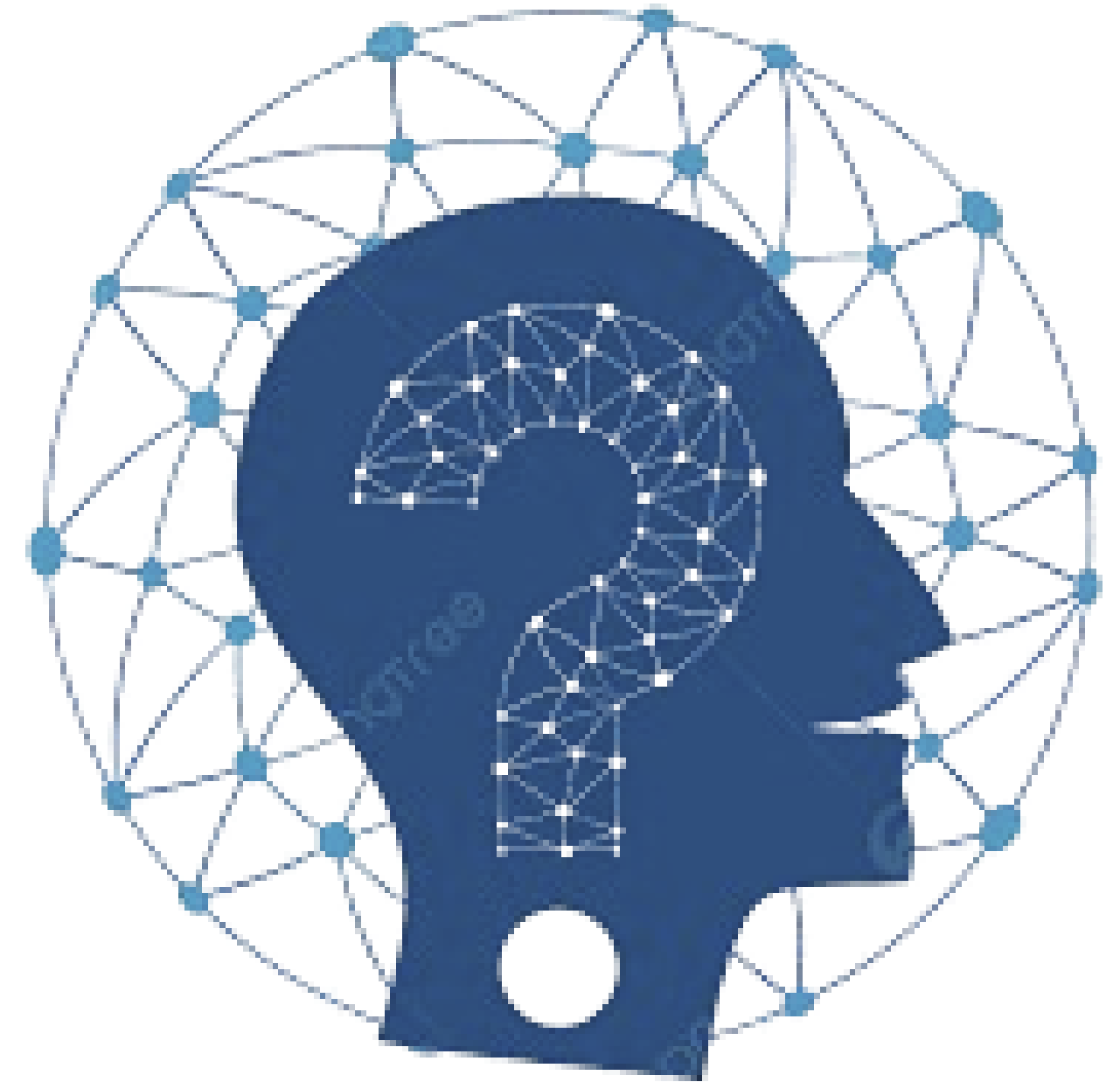
What Problem Is This Paper Trying to Solve?

Large Neural Networks: Modern deep neural networks achieve high accuracy using millions of parameters. This makes the models very large and resource-intensive.

Deployment Difficulty: Large models are designed to run on GPUs and high-end servers. Running them on mobiles or embedded devices becomes difficult.

Inefficient Resource Usage: Not all parameters contribute equally to the network's performance. Many weights consume resources while adding little value.

Main Objective: The aim is to reduce model size and computation cost. This should be achieved without significantly reducing accuracy.



Why Is Neural Network Compression Necessary?



Memory Bottleneck

- Accessing memory consumes more energy than computation itself.
- Large models slow down inference due to frequent memory access.



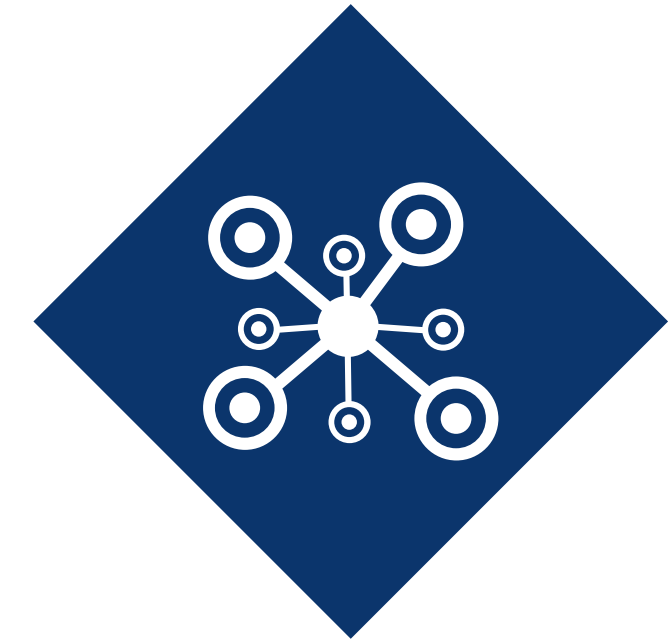
Embedded System Constraints

- Devices like phones and IoT systems have limited RAM and storage.
- Large neural networks are inefficient on such platforms.



Real-Time Performance Needs

- Applications like vision and speech need fast responses.
- Smaller models allow quicker and more efficient inference.



Scalability and Deployment

- Compressed models are easier to deploy and update.
- They are more practical for large-scale real-world applications.

Earlier Approaches to Neural Network Compression

Earlier compression methods focused on removing redundancy in trained networks to reduce size and computation while preserving accuracy.

Pruning

Pruning removes unimportant weights or neurons from a trained network. It reduces model size but usually requires fine-tuning to maintain accuracy.

Quantization

Quantization lowers the precision of weights and activations (e.g., FP32 to INT8). This reduces memory usage and speeds up inference, with possible accuracy loss.

Low-Rank Factorization

Large weight matrices are decomposed into smaller low-rank matrices. This reduces parameters but may add extra computation steps.

Knowledge Distillation

A small student network learns from a large teacher network. This helps achieve good accuracy with fewer parameters.

Weight Sharing

Multiple connections share the same parameters to reduce memory usage. Too much sharing can limit model performance.



Limitations of Earlier Methods



Heavy Dependence on Manual Tuning

- Pruning thresholds, quantization bit-widths, and rank selection had to be hand-crafted
- Performance depended strongly on expert knowledge
- Not easily reproducible across models or datasets



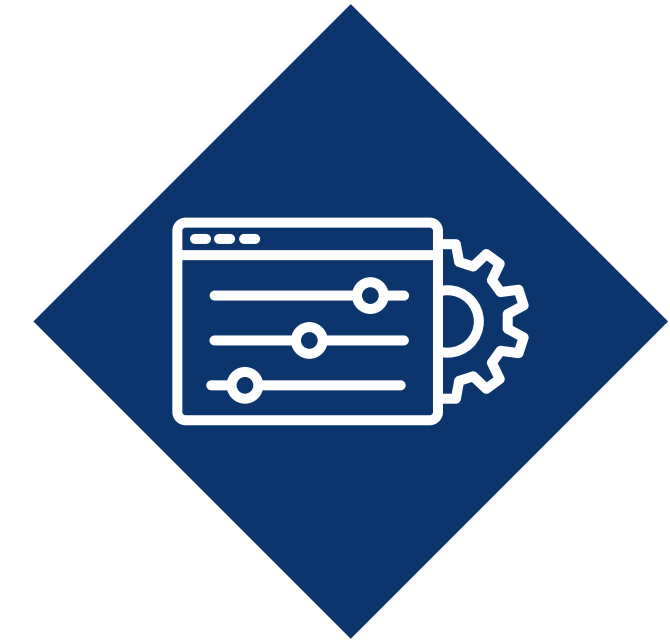
Accuracy-Compression Trade-off

- Higher compression often caused significant accuracy degradation
- Sensitive layers (e.g., early convolution layers) were hard to compress
- No principled way to guarantee performance preservation



Poor Scalability to Deep and Modern Architectures

- Designed mainly for:
 - Shallow CNNs
 - Fully connected networks
- Struggled with:
 - Very deep models
 - Transformers
 - Large-scale datasets



Lack of Joint Optimization

- Compression was decoupled from learning
- Models were not trained with compression constraints in mind
- Led to sub-optimal compressed architectures

DEEP NEURAL NETWORK

How machine learns complex patterns

A Deep Neural Network learns complex patterns by adjusting weights across multiple hidden layers to minimize prediction error.

Input Layer

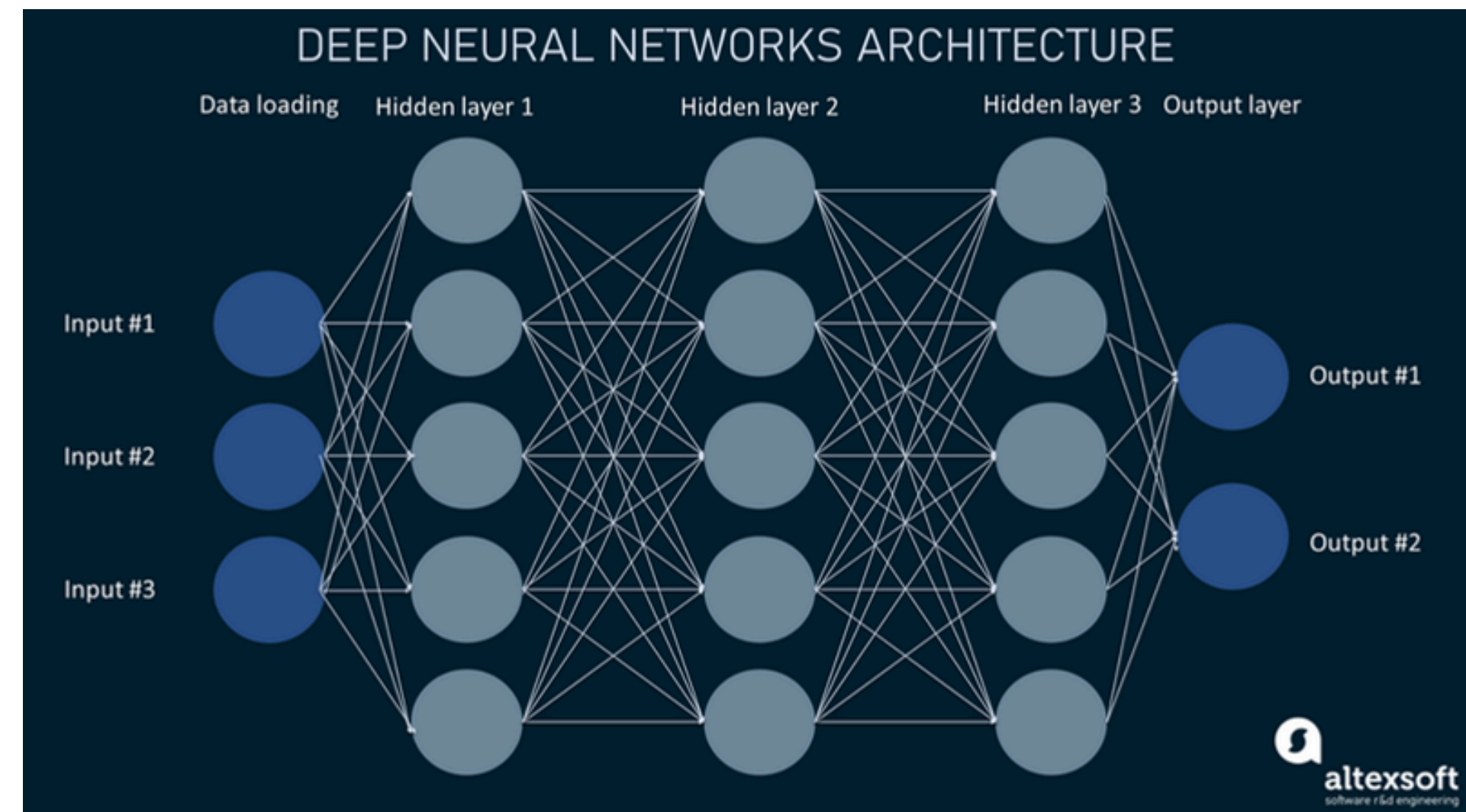
- Images
- Text
- Audio
- Numerical data

Hidden Layers

- Multiple processing layers
- Learn features automatically
- Apply non-linear transformations

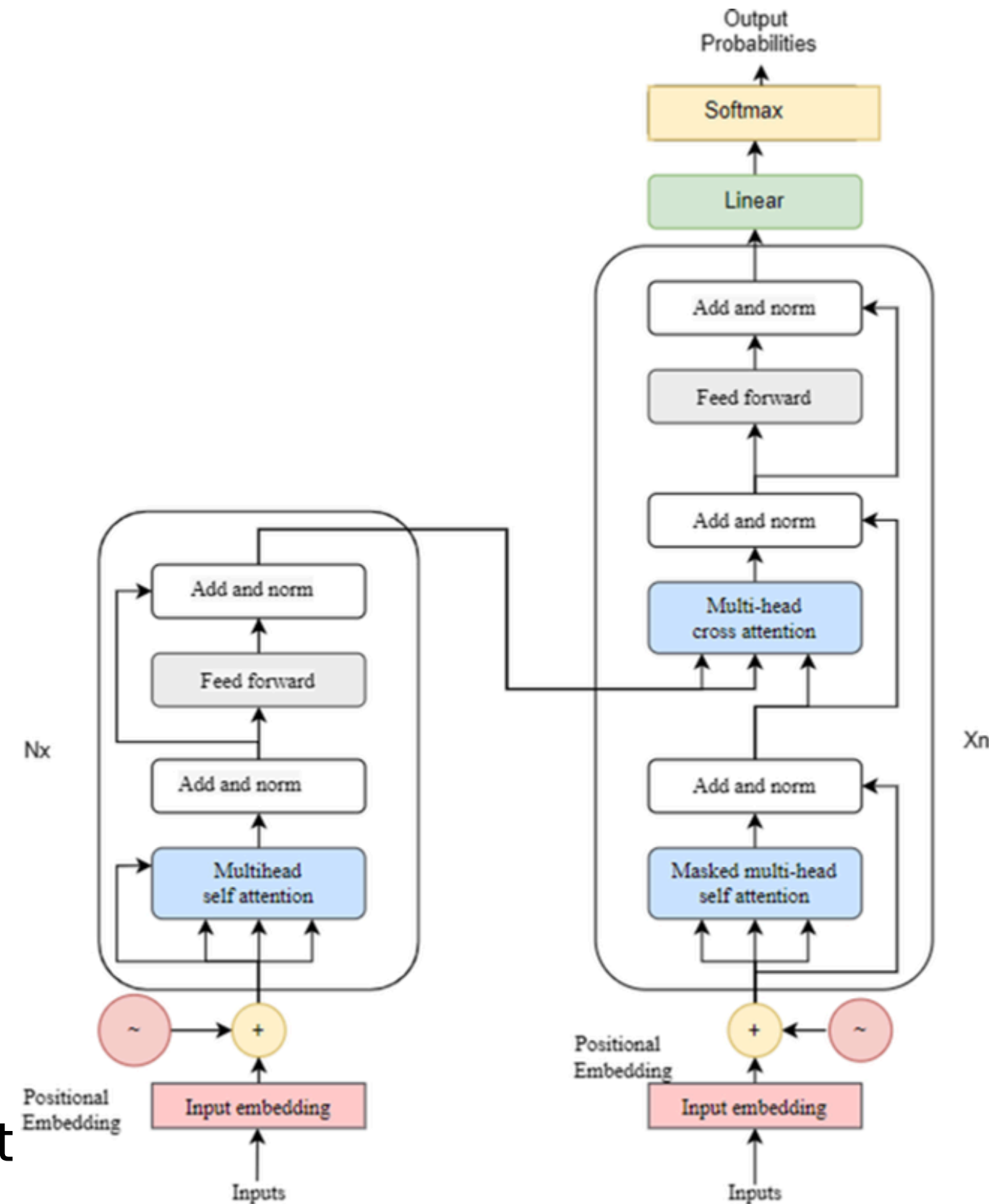
Output Layer

- Classification
- Prediction
- Decision



Challenges of running DNNs on Embedded System

- **Limited Computational Power:** Embedded devices lack GPUs and high-performance CPUs.
- **Memory Constraints :** DNN models require large RAM and storage for weights and activations.
- **High Energy Consumption :** Continuous inference drains battery-powered devices quickly.
- **Latency Requirements :** Real-time applications need fast inference with minimal delay.
- **Model Size :** Large models are difficult to deploy on resource-constrained hardware.
- **Thermal Constraints :** Intensive computation causes overheating in compact systems.
- **Hardware Compatibility :** Limited support for floating-point operations and deep learning libraries



Overview of the Deep Compression Pipeline

The paper proposes a solution called Deep Compression, which is a systematic method to **compress deep neural networks while preserving their original accuracy.**

Deep Compression is a **three-stage method** used to reduce the size of deep neural networks.

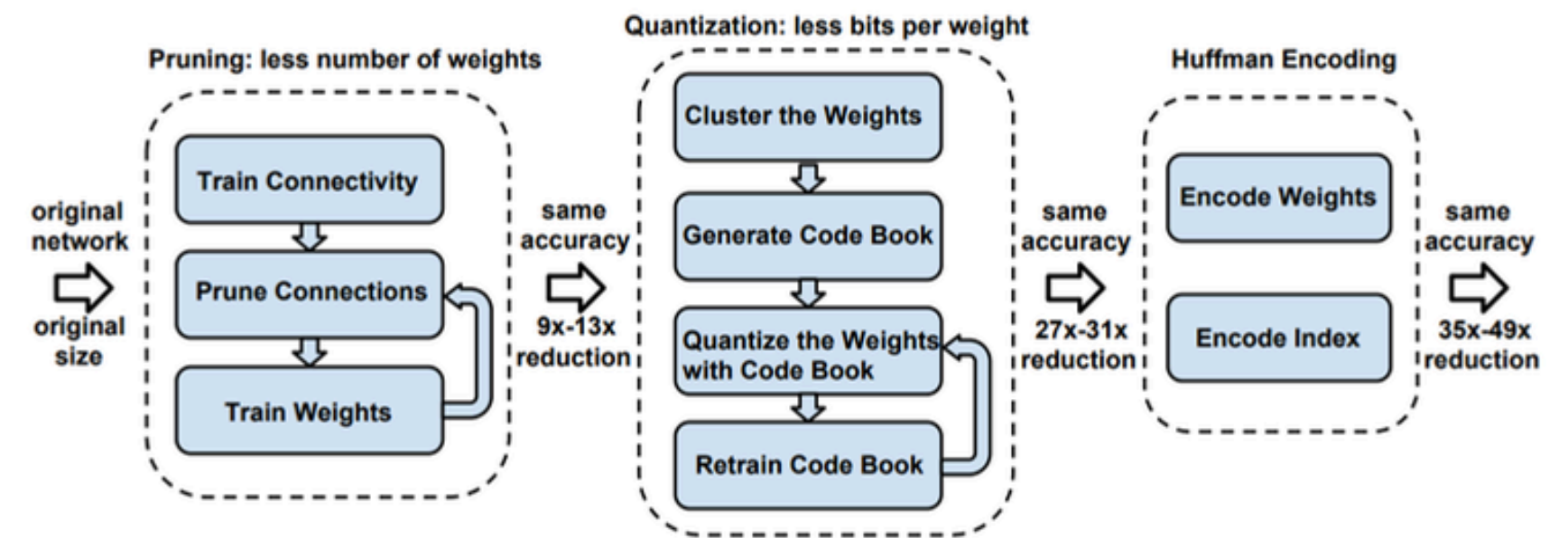


Figure: Three-stage Deep Compression pipeline showing progressive reduction in model size without accuracy loss.

Stage 1: Pruning

It removes unnecessary connections, reducing the total number of parameters

Stage 2: Quantization and weight sharing

It reduces the number of bits needed to store each weight

Stage 3: Huffman coding

It exploits redundancy to further compress the stored weights and indices

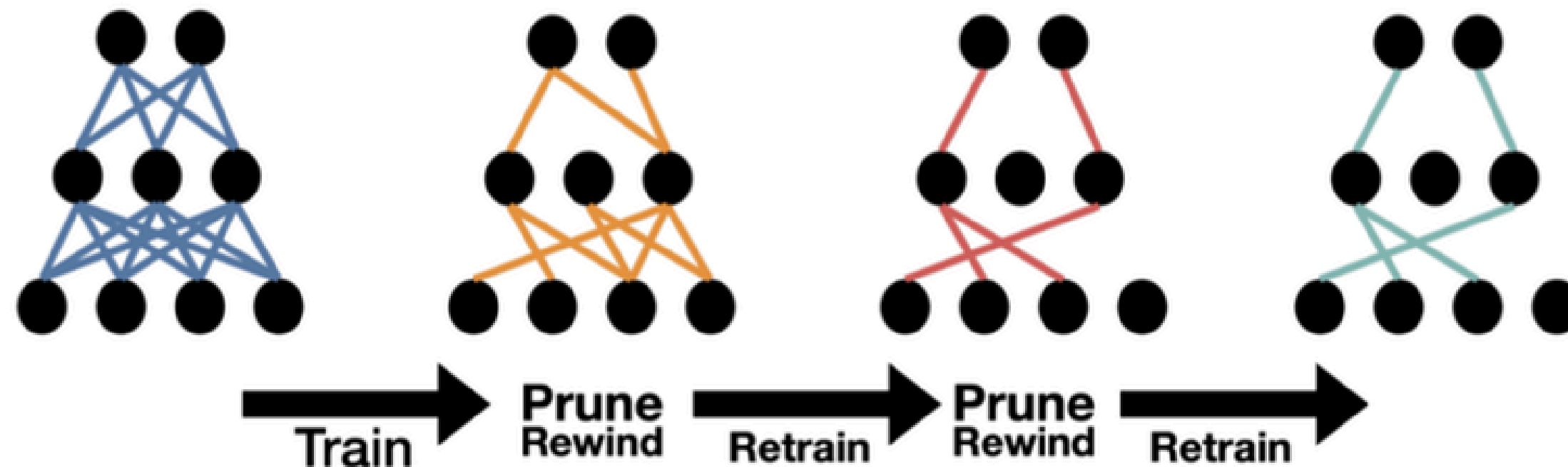
These stages work together to **achieve high compression ratios** and the final compressed model maintains the **same accuracy** as the original network

STAGE 1: NETWORK PRUNING

In a trained deep neural network, not all connections are equally important. Many weights have very small magnitudes, meaning they contribute very little to the final output of the network.

Network pruning is the process of **removing** these **small-weight connections** while keeping the important connections intact.

This reduces the number of parameters without reducing accuracy.



Conceptual illustration of magnitude-based network pruning (used to explain the pruning idea).

How Pruning is done?

The pruning process follows three clear steps:

Step 1: Train the original network

- The network is first trained normally using standard backpropagation.
- At this stage, the network learns:

- Both weights
- And connectivity between neurons

This is called learning the connectivity.



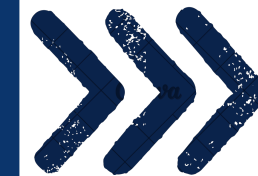
Step 2: Prune small-weight connections

- After training, all weights are examined.
- A threshold value is chosen.
- Any connection whose weight magnitude is below the threshold is removed.

Mathematically:

$$\text{If } |w_{ij}| < \epsilon \Rightarrow w_{ij} = 0$$

These weights are set to zero, effectively removing the connection.



Step 3: Retrain the pruned network

- After pruning, the network becomes sparse.
- Only the remaining connections are retrained.
- Pruned connections remain permanently removed (their gradients are masked).

This retraining allows the remaining weights to compensate for the removed ones.

Why accuracy is not lost ?

Even though many connections are removed:

- The network is still over-parameterized
- **Important connections remain**
- Retraining fine-tunes the surviving weights

As a result:

- **Accuracy is fully recovered**
- Sometimes even overfitting is reduced

Span Exceeds $8=2^3$

idx	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
diff		1			3								8			3
value		3.4			0.9								0			1.7

Filler Zero

Sparse representation after pruning

Representing the matrix sparsity with relative index.
Padding filler zero to prevent overflow.

After pruning, the weight matrix becomes sparse (mostly zeros).

Instead of storing all weights, the paper stores:

- **Only non-zero weights**
- Their indices

This is done using:

- Compressed Sparse Row (CSR) or
- Compressed Sparse Column (CSC) formats

This **reduces memory** and prepares the model for further compression.

Relative Indexing

To reduce storage further:

- Instead of storing absolute indices, the paper stores index differences
- These differences are encoded using:
 - 8 bits for convolution layers
 - 5 bits for fully connected layers

If the index difference exceeds the allowed range:

- A filler zero is inserted

This prevents overflow and keeps indexing compact.

STAGE 2: Quantization and Weight sharing

After pruning, the remaining weights are still stored as 32-bit floating-point values, which is inefficient for:

- Memory storage
- Memory bandwidth
- Energy consumption (especially on mobile/embedded devices)

Key Idea : Instead of storing every remaining weight independently:

- Force multiple connections to share the same weight value
- Store a small index instead of a full 32-bit float

32 bits → 5–8 bits

What we achieve:

- Reduces bits per weight from 32 bits → 5–8 bits
- Achieves 3×–4× additional compression on top of pruning
- Minimal loss in accuracy when done correctly

Given k clusters, we only need **$\log_2(k)$ bits to encode the index**. In general, for a network with n connections and each connection is represented with b bits, constraining the connections to have only k shared weights will result in a compression rate of:

$$r = \frac{nb}{n\log_2(k) + kb}$$

How quantization works?

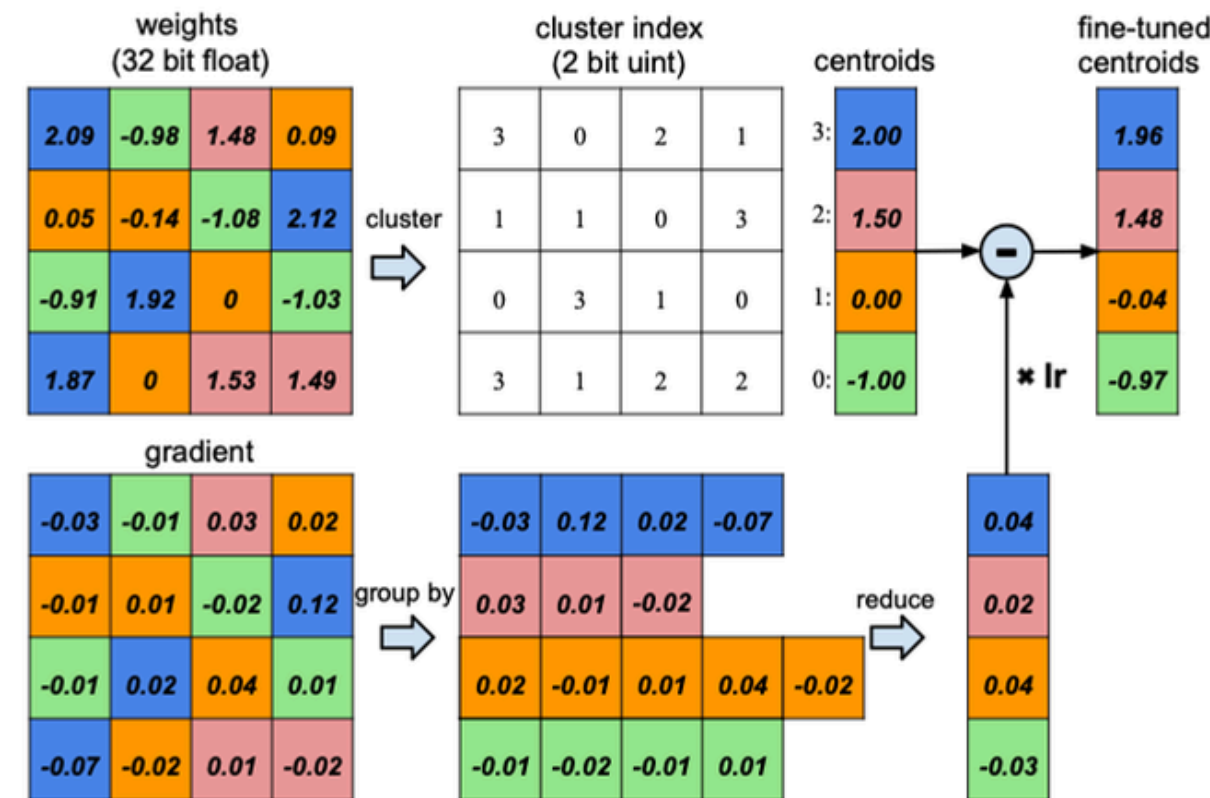
The quantization process is done in two steps:

Step 1: Weight clustering

- We cluster the pruned weights using k-means and then store the cluster centroids and the indices of each cluster as 2-bit integers.

Mathematically, clustering minimizes:

$$\arg \min_C \sum_{i=1}^k \sum_{w \in c_i} |w - c_i|^2$$



Finally we just apply !

$$c_k^{new} = c_k^{old} - \eta \cdot \frac{\partial L}{\partial c_k}$$

Step 2: Gradient Computation

- We calculate the gradient matrix of the pruned weights, same shape as the weights matrix.
- Gradients are grouped by clusters, all gradients belonging to the same cluster are summed and stored in a single array.

This corresponds to:

$$\frac{\partial L}{\partial c_k} = \sum_{i,j} \frac{\partial L}{\partial W_{ij}} \text{ where } W_{ij} \in \text{cluster } k$$

Why summation makes sense (not averaging)
Each gradient tells:

"If this weight were free, move it this way."

Since the centroid represents all of them, we:

- Add up their "opinions"
- Move the centroid in the direction that reduces total loss

Initialising Shared Weight Cluster Means – Three methods

Why it matters?

- Weight sharing uses k-means clustering to group similar weights
- Initial placement of centroids strongly affects:
 - Quality of clustering
 - Quantization error
 - Final network accuracy
- Poor initialization can cause:
 - Important weights to be badly approximated
 - Irreversible accuracy loss even after fine-tuning

Method 1: Forgy (Random) Initialization

- Randomly selects k existing weights as initial centroids
- Simple and fast
- Problem
- Centroids cluster around dense regions (peaks)
- Few centroids represent large-magnitude weights
- Leads to poor approximation of important weights

Method 2: Density-Based Initialization

- Uses the CDF of weights
- Centroids placed by:
 - Uniform spacing along CDF (y-axis)
 - Mapping back to weight values (x-axis)

Pros

- More centroids near high-density regions
- Better than random initialization

Cons

- Still under-represents large-magnitude weights
- Sensitive to weight distribution shape

Method 3: Linear Initialization (Best Method)

- Centroids evenly spaced between:
 - Minimum and maximum weight values
- Independent of weight distribution

Advantages

- Ensures representation of large-magnitude weights
- Prevents dominant weights from being poorly clustered
- Most robust across layers and networks

Stage 3: HUFFMAN CODING

What it is?

- Lossless compression using variable-length prefix codes
- Frequent symbols → fewer bits, rare symbols → more bits

Why it works here?

- After pruning & quantization, distributions are non-uniform
- Weight indices and sparse index differences are heavily biased

What is encoded?

- Quantized weight indices
- Sparse matrix relative index differences
- Applied after training, no accuracy impact

Impact

- Additional 20–30% size reduction
- Final compression: 35× (AlexNet), 49× (VGG-16)
- Removes statistical redundancy with zero loss

Experimental Results

Overall Achievement

- 35× to 49× compression with zero accuracy loss
- Pruning alone: 9× – 13× reduction
- Pruning + Quantization: 27× – 31× reduction
- All three stages: 35× – 49× reduction

Compression Results by Network

- LeNet-300-100: 1070 KB → 27 KB (40× compression)
- LeNet-5: 1720 KB → 44 KB (39× compression)
- AlexNet: 240 MB → 6.9 MB (35× compression)
- VGG-16: 552 MB → 11.3 MB (49× compression)

Accuracy Preservation

- All networks: Zero accuracy loss maintained
- Some networks show slight accuracy improvement
- AlexNet: 42.78% error maintained perfectly
- VGG-16: 31.50% → 31.17% (improved by 0.33%)

Experimental Results

Quantization Precision

- CONV layers: 8-bit quantization (256 shared weights)
- FC layers: 5-bit quantization (32 shared weights)
- Result: Zero accuracy loss with reduced bit precision
- More aggressive 4/2-bit causes 1.99% accuracy drop

Layer Compression Details

- Fully connected layers: Most heavily compressed (90%+ of model)
- VGG-16 fc6 layer: Reduced from 103M to ~1.6M parameters
- Average weight bits: Reduced from 32-bit to 4-5 bits
- Codebook overhead: Negligible (< 1% of total)

Key Insight

- Model now fits in on-chip SRAM instead of off-chip DRAM
- Enables practical deployment on mobile devices
- Critical for real-time inference applications

Benefits of Deep Compression

• Real-World Impact

Performance:

- 3–4× speedup (all devices)
- 4.2× speedup (mobile GPU)

Energy (CRITICAL):

- 7× less energy on CPU
- 3.3× less on GPU
- 4.2× less on mobile

Mobile Deployment:

- AlexNet: 6.9MB (fits in app budget)
- App Store limit: 100MB auto-download
- Now feasible for smartphones

Technical Insight:

- DRAM 700× more expensive than computation
- Compressed model in SRAM saves massive energy
- Perfect for battery-constrained devices

Why Deep Compression Is Effective

Individual Methods Fail:

- Pruning alone: Max 8× compression
- Quantization alone: Max 8× compression
- Accuracy drops beyond these limits

Combined = Success:

- Pruning + Quantization + Huffman: 35-49×
- ZERO accuracy loss
- Synergy makes difference

Root Cause - Over-Parameterization:

- 92% of weights are redundant
- LeNet: 92% redundancy
- VGG-16: 92.5% redundancy

How It Works:

1. Pruning removes 89% of weights
2. Remaining weights easier to quantize
3. Huffman encoding exploits distribution
4. Retraining recovers accuracy

Why Revolutionary:

- Previous methods: compress OR maintain accuracy
- Deep Compression: compress AND maintain accuracy
- 35-49× unprecedented at the time

Effect of Compression on Accuracy

Accuracy vs Compression Rate

Different compression methods were compared:

1. Pruning only
2. Quantization only
3. Pruning + Quantization
4. SVD

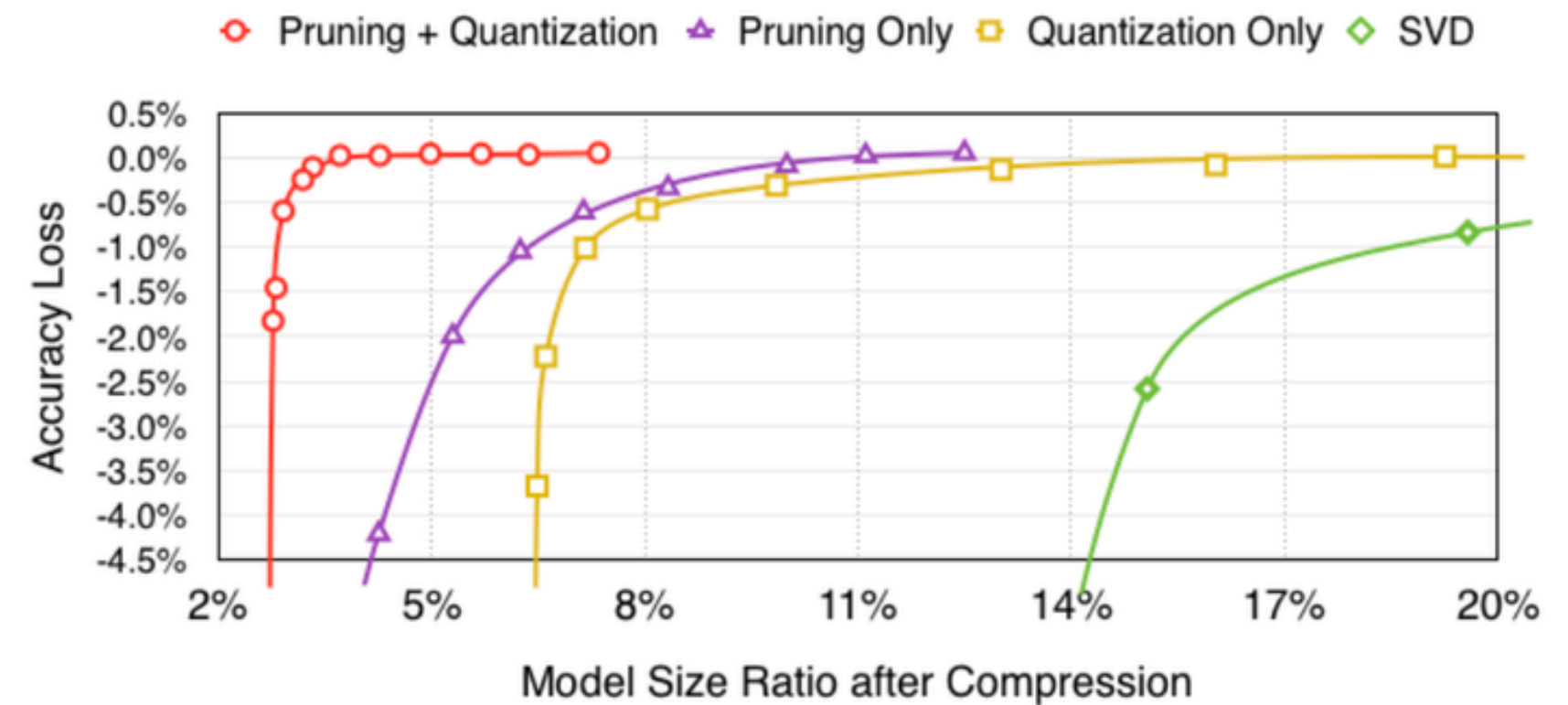
When used individually:

- **Pruning** → accuracy drops below ~8% model size
- **Quantization** → accuracy drops below ~8% model size

When combined:

- Pruning + Quantization compresses the model to ~3% of original size
- No loss in accuracy

SVD gives lower compression and higher accuracy loss



Accuracy v.s. compression rate under different compression methods. Pruning and quantization works best when combined.

Applications of Deep Neural Networks (with Deep Compression)

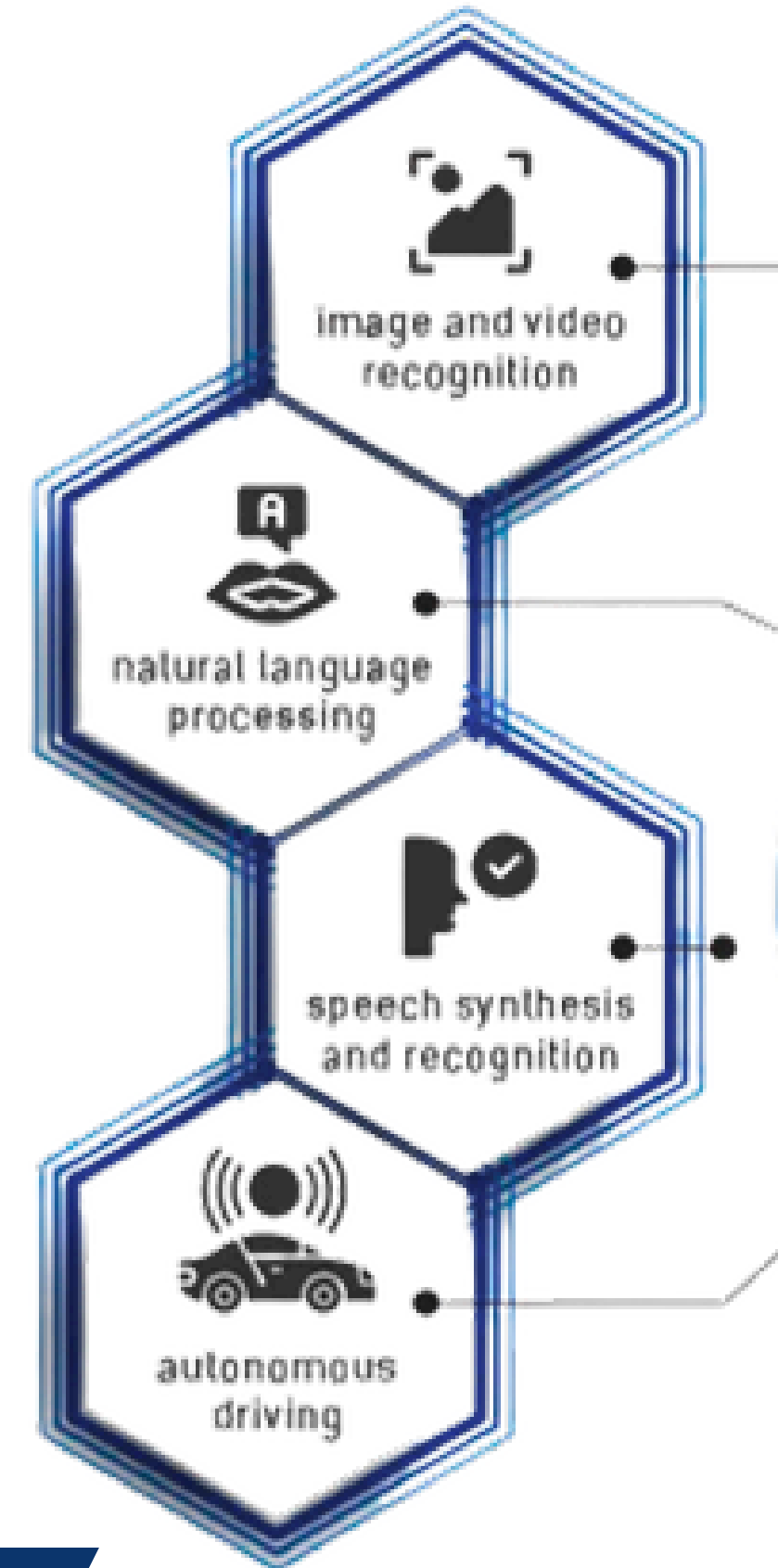
- Deep neural networks are widely **used in computer vision tasks** such as **image classification**, **object detection**, and **face recognition**
- Due to Compression, large models like AlexNet and VGG-16 can now be deployed on mobile and embedded devices
- Compressed models **fit into on-chip SRAM**, avoiding expensive DRAM access
- This enables real-time inference for latency-critical applications

Useful in applications such as:

- **Mobile apps** with size and download limits
- **Autonomous vehicles** (e.g., pedestrian detection)
- **IoT and edge devices** with limited memory and power

Key benefit:

smaller model size, faster execution, and lower energy consumption



Limitations and Future Work

Limitations:

- Current CPU and GPU libraries do not fully support:
 - Indirect indexing
 - Relative indexing in sparse + quantized models
- Because of this, quantized models could not be fully benchmarked on standard hardware
- Performance gains are reduced when batching is allowed, since memory reuse improves naturally

Future Work:

- Develop custom GPU kernels to support sparse and quantized networks
- Design specialized hardware accelerators (ASICs) for compressed models
- Allow flexible custom quantization bit-widths
- Shift energy usage from off-chip DRAM to on-chip SRAM, improving efficiency

Conclusion

This work presents **Deep Compression**, a three-stage pipeline:



- The method achieves **35× compression for AlexNet** and **49× compression for VGG-16**
- Compression is achieved **without any loss in accuracy**
- **Reduced model size** allows networks to fit in on-chip SRAM, lowering memory access cost
- Deep Compression makes it feasible to deploy large neural networks on mobile and embedded systems

Overall impact:

faster inference, lower energy consumption, and practical deployment of deep learning on edge devices